

A Fast Algorithm for Generating All Triangulations of Biconnected Plane Graphs

Tawkir Aziz Rahman

CSE, BUET

Overview

- 1 Introduction
- 2 Our Approach
- 3 Algorithm
- 4 Proofs
 - Proof of Completeness
 - Existence of Valid Root Triangulation
 - Time Complexity Analysis
 - Space Complexity Analysis
- 5 Conclusion

The Problem:

- Generate **all possible triangulations** of biconnected plane graphs
- Previous methods fail with complex structures due to multi-edge creation
- Important applications in graphics, engineering, and chip design

Our Contribution:

- Novel algorithm with **constant time per triangulation**
- Solves the multi-edge problem using **safe root** concept
- Practical for real-world applications

Our Approach: Key Innovations

Novel Tree Structure:

- Instead of a single global genealogical tree, we use **local genealogical trees** for each face/cycle
- Each face maintains its own tree of triangulations with parent-child relationships
- Enables independent processing of faces while avoiding multi-edge issues

Safe Root Concept:

- Introduces **safe root vertex selection** to prevent multi-edge creation
- Uses incremental chord checking to find optimal root vertex position
- Ensures all initial chords from root vertex are valid and non-conflicting

Our Approach: Recursive Tree Construction

Local Tree Generation:

- For each face F_i , build a genealogical tree $\mathcal{T}(F_i)$
- Initialize with safe root triangulation where all chords emanate from safe root vertex
- Generate children by flipping generating chords while maintaining parent-child relationships

GlobalChordSet Management:

- Maintain global set of all chords used across all faces
- Before adding any chord, check if it already exists in GlobalChordSet
- Prevents multi-edge creation by rejecting conflicting chords
- Dynamic addition/removal during recursive exploration

Our Approach: Combinatorial Integration

Recursive Face Processing:

- Process faces sequentially from F_1 to F_k
- For each partial triangulation T , recursively triangulate remaining faces
- Combine local triangulations T_i to form complete graph triangulation $T + T_i$

Efficient Output Generation:

- Output complete triangulation only when all faces are processed ($i = \text{face.length}$)
- For intermediate states, recursively continue to next face
- Ensures only valid complete triangulations are output

Constant-Time Operations:

- Safe root finding in $O(1)$ time per face chord operations and GlobalChordSet updates in constant time
- Overall $O(1)$ time per triangulation generation

Our Approach: Algorithm Overview

Start with Biconnected Graph

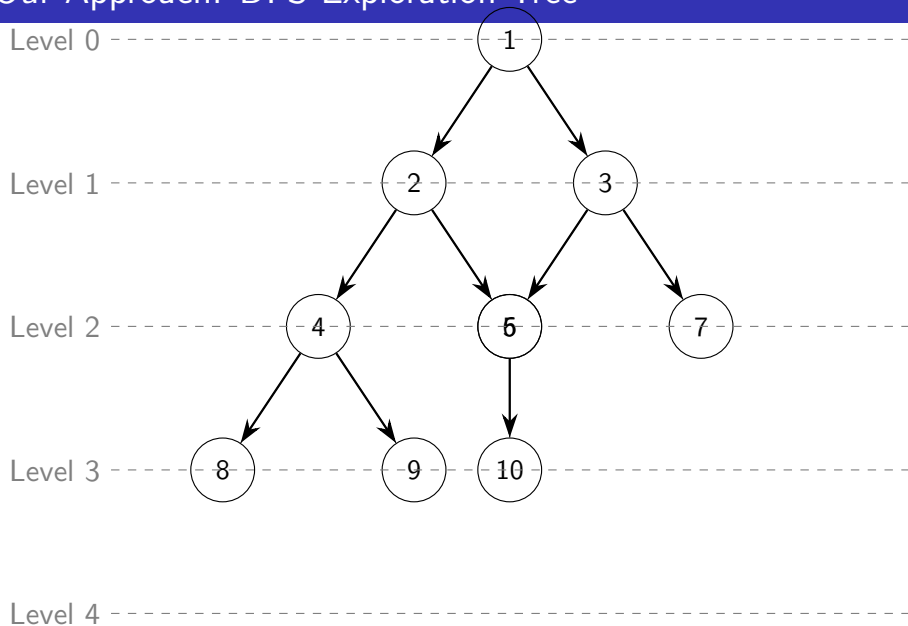


Decompose into Faces $F_1, F_2, \dots, F_k$

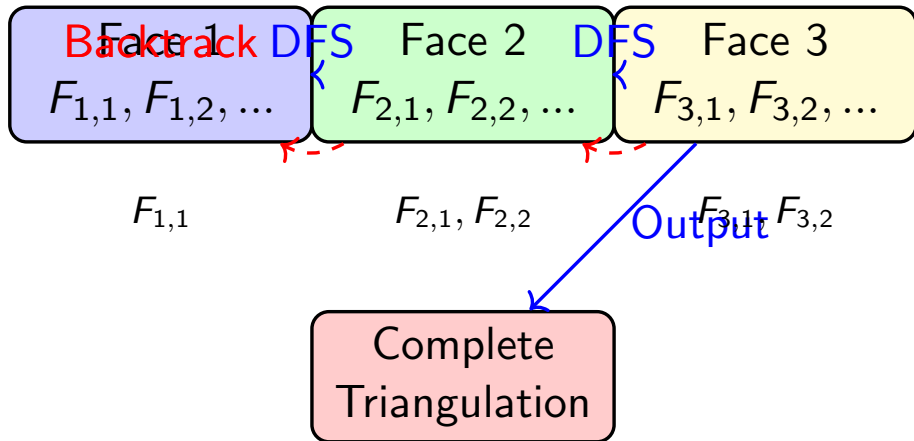


Find Safe Root for Each Face

Our Approach: DFS Exploration Tree

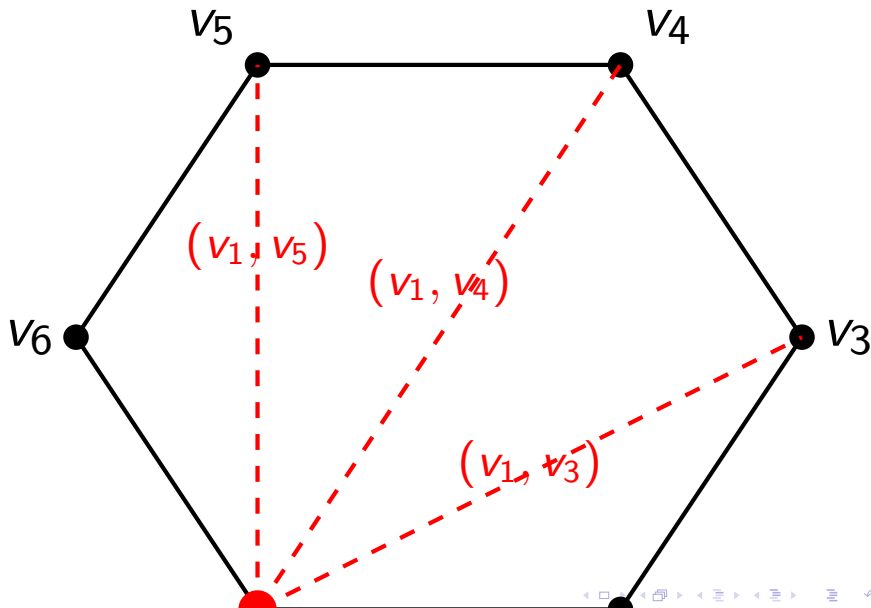


Our Approach: Face Processing Workflow

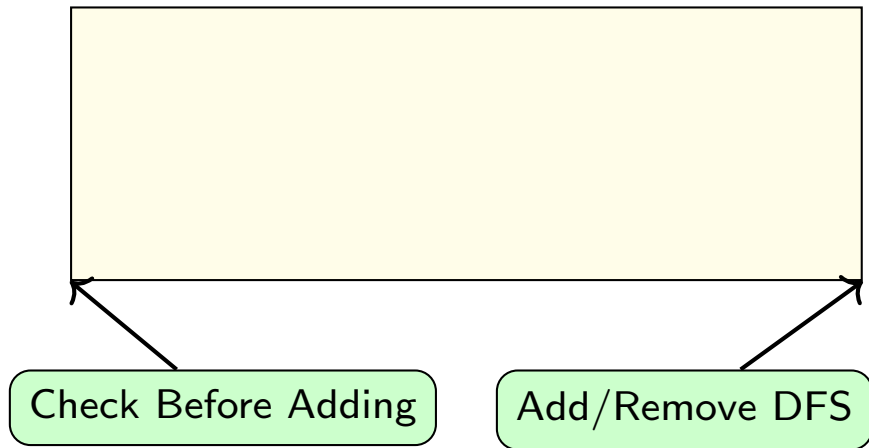


Process: For each triangulation of Face i , recursively find all compatible triangulations for Face $i + 1$

Our Approach: Safe Root Concept Visualization

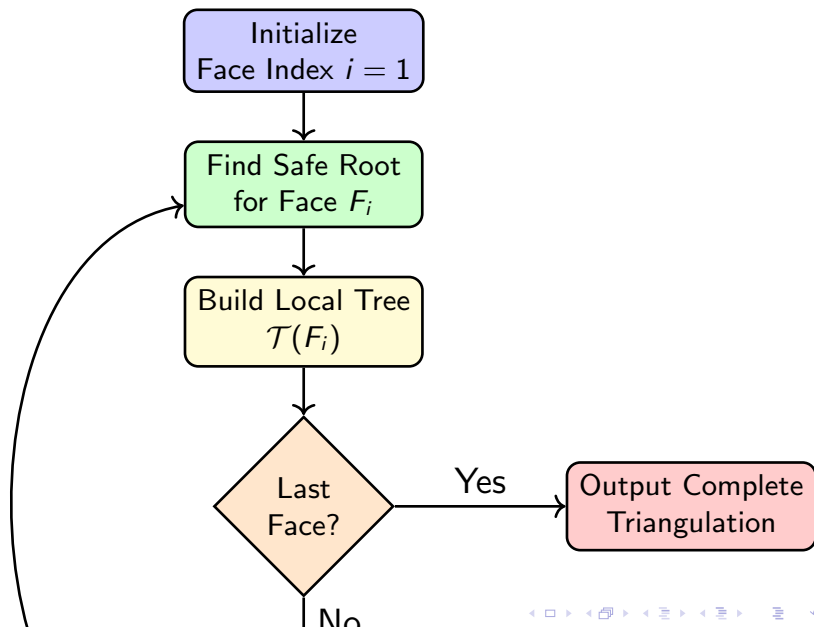


GlobalChordSet



Purpose: Prevents multi-edge creation by tracking all chords used across faces

Our Approach: Complete Algorithm Flow



Algorithm Structure Overview

Our algorithm consists of 5 main procedures:

- ➊ **StartTriangulation(G, k)**
 - Entry point - initializes the process
- ➋ **FindAllTriangulations(F, T, i)**
 - Generates all triangulations for face F_i
 - Uses safe root as starting point
- ➌ **FindAllChildTriangulations(T_i, T, i)**
 - Explores genealogical tree via chord flipping
- ➍ **FindSafeRoot(F, i)**
 - Finds safe root vertex and creates initial triangulation
- ➎ **OutputTriangulation(T, T_i, i)**
 - Outputs complete triangulation or continues to next face

Find All Child Triangulations

Input: T_i = current triangulation, T = partial solution, i = face index

```
1: procedure FINDALLCHILDTRIANGULATIONS( $T_i, T, i$ )
2:   OUTPUTTRIANGULATION( $T, T_i, i$ )
3:   if  $T_i$  has no generating chords then
4:     return
5:   end if
6:   Let  $(v_b, v_{b'})$  be the leftmost blocking chord of  $T_i$ 
7:   for all vertices  $v_j$  where  $j \geq b$  do
8:     if  $(v_1, v_j)$  is a chord of  $T_i$  or  $(v_1, v_j) \notin \text{GlobalChordSet}$  then
9:       Add chord  $(v_1, v_j)$  to GlobalChordSet
10:      FINDALLCHILDTRIANGULATIONS( $T_i(v_1, v_j), T, i$ )
11:      Remove chord  $(v_1, v_j)$  from GlobalChordSet
12:    end if
13:  end for
14: end procedure
```

Find All Triangulations for a Face

Input: F = faces array, T = partial solution, i = current face index

```
1: procedure FINDALLTRIANGULATIONS( $F, T, i$ )
2:    $n \leftarrow$  number of vertices in face  $F_i$ 
3:    $T_{ir} \leftarrow$  FINDSAFEROOT( $F, i$ )                                ▷ Time:  $O(1)$  per triangulation
4:   Add all chords of  $T_{ir}$  to  $GlobalChordSet$                         ▷ Time:  $O(n)$ 
5:   OUTPUTTRIANGULATION( $T, T_{ir}, i$ )
6:    $T_i \leftarrow T_{ir}$ 
7:   for  $j = n - 1$  down to 3 do
8:     if  $(v_1, v_j) \notin GlobalChordSet$  then
9:       Add chord  $(v_1, v_j)$  to  $GlobalChordSet$ 
10:      FINDALLCHILDTRIANGULATIONS( $T_i(v_1, v_j), T, i$ )
11:      Remove chord  $(v_1, v_j)$  from  $GlobalChordSet$ 
12:    end if
13:  end for
14:  Remove all chords of  $T_{ir}$  from  $GlobalChordSet$                     ▷ Time:  $O(n)$ 
15: end procedure
```

Output Triangulation

Input: T = partial solution, T_i = current face triangulation, i = face index

```
1: procedure OUTPUTTRIANGULATION( $T, T_i, i$ )  
2:   if  $i = \text{totalFaces}$  then  
3:     Output complete triangulation ( $T + T_i$ )  
4:   else  
5:     FINDALLTRIANGULATIONS( $F, T + T_i, i + 1$ )  
6:   end if  
7: end procedure
```

Note: This recursive structure combines triangulations from individual faces to generate complete graph triangulations.

Find Safe Root

Input: F = faces array, i = face index

```
1: procedure FINDSAFEROOT( $F, i$ )
2:    $v_{root} \leftarrow \text{FINDSAFEROOTVERTEX}(F, i)$ 
3:   Create triangulation by adding chords from  $v_{root}$  to all non-neighbors
4:   return root triangulation  $T_{ir}$ 
5: end procedure

1: procedure FINDSAFEROOTVERTEX( $F, i$ )
2:    $n \leftarrow$  number of vertices in face  $F_i$ 
3:    $startIndex \leftarrow n - 1$ 
4:    $endIndex \leftarrow 0$ 
5:   while  $startIndex > endIndex + 1$  do
6:     if chord exists between  $v_{startIndex}$  and  $v_{endIndex}$  in GlobalChordSet then
7:        $startIndex \leftarrow startIndex - 1$ 
8:     else
9:        $endIndex \leftarrow endIndex + 1$ 
10:    end if
11:  end while
12:  return  $v_{endIndex}$  as safe root vertex
13: end procedure
```

Main Algorithm - Start Triangulation

Input: G = biconnected plane graph, k = number of faces

- 1: **procedure** STARTTRIANGULATION(G, k)
- 2: Label the faces as $F_1, F_2, \dots, F_k$ arbitrarily
- 3: Initialize $GlobalChordSet \leftarrow \emptyset$
- 4: FINDALLTRIANGULATIONS($F, \emptyset, 1$)
- 5: **end procedure**

Note: This is the entry point of the algorithm. It processes faces sequentially, building up complete triangulations through recursive exploration.

Proof of Completeness: Pruning Strategy

Pruning Operations:

- Algorithm prunes triangulations where blocking chord is a duplicate chord
- Duplicate chord = chord that occurred in earlier faces
- Prune entire branch when duplication detected

Theorem (Completeness via Pruning):

All children of a pruned branch would have been invalid triangulations.

Key Insight:

- Need to prove: blocking chord is never flipped in children
- If blocking chord persists in all children, and it's a duplicate
- Then all children are invalid, so pruning is safe

Proof of Completeness: Blocking Chord Analysis

Setup:

- Let chord $a-b$ be a blocking chord in triangulation tree node
- Two points form triangles with $a-b$ chord on both sides:
 - c : point on opposite side of root vertex
 - r : point on root side

Three Cases Based on Position of r :

- 1 r is on the left of root vertex
- 2 r is on the right of root vertex
- 3 r is at the root

Proof of Completeness: Case 1

Case 1: r is on the left of the root vertex

- Flipping $a-b$ chord $\Rightarrow$ results in $r-c$ chord
- Before flipping: $b-r$ chord was also a blocking chord
- $b-r$ positioned to the left of $a-b$
- Therefore: $a-b$ could not be leftmost blocking chord
- After flip: $r-b$ chord still exists and remains left of $c-r$
- Consequence: new $r-c$ chord cannot be leftmost blocking chord

Conclusion:

Flipping $a-b$ blocking chord is NOT permitted by algorithm in this case.

Proof of Completeness: Cases 2 & 3

Case 2: r is on the right of the root vertex

- Flipping $a-b$ chord $\Rightarrow$ produces $r-c$ chord
- However: $a-r$ chord is more to the left than $r-c$ chord
- This flip is invalid according to the algorithm

Case 3: r is at the root

- Flipping $a-b$ chord $\Rightarrow$ creates root- c chord
- This gives child triangulation's root vertex better visibility than parent
- Violates the algorithm's constraints

Final Conclusion:

In all three cases, flipping a blocking chord is impossible. If blocking chord is duplicate, all children are invalid. Pruning is safe and complete.

Existence of Valid Root: Definitions

Definition (Root Triangulation):

Root of the tree of triangulations, constructed by selecting a root vertex and adding chords from this root vertex to all non-neighboring vertices of the current face.

Definition (Safe Root Vertex):

A vertex from which adding chords to every other non-neighboring node will NOT create a multi-edge problem. None of the previous faces share a chord where one endpoint is the root vertex and the other is a vertex of the current face.

Existence of Valid Root: Main Theorem

Theorem (Existence of Safe Root):

In any face of a biconnected planar graph, there always exist at least two safe vertices that can serve as root vertices for valid triangulations.

Proof Strategy:

- Consider a face with n vertices, numbered 1 to n
- Since planar graph, chords can only exist in outer plane
- Analyze two cases:
 - 1 No chords from previous faces
 - 2 At least one chord from previous faces

Existence of Valid Root: Case 1

Case 1: No chords from previous faces

- If no chords from previous faces with both endpoints inside current face
- Then all n vertices are eligible to be safe vertices

Simple but important base case!

Existence of Valid Root: Case 2 Setup

Case 2: At least one chord from previous faces

- Suppose chord $a-b$ on outer plane from previous face
- Both a and b are disqualified from being safe vertices
- Due to planarity: chords cannot cross each other
- Since $a-b$ chord exists, no chord on outer plane can overtake it

Consequence:

- Nodes $(a + 1, a + 2, \dots)$ might have connections between them
- They CANNOT have chords with nodes $(a - 1, \dots, b + 1)$
- (Would violate planarity by crossing $a-b$ chord)

This creates T_n structure:

- n = number of internal vertices between endpoints of constraining chord

Current face contains two complementary T_n structures:

- ① Structure 1: $a, a + 1, \dots, b$
- ② Structure 2: $a, a - 1, \dots, b$

Internal vertices: numbered $1, \dots, n - 3$

Range:

- Minimum: T_1 (since T_0 would mean a and b are neighbors)
- Maximum: T_{n-3} (accounting for endpoints and complementary structure)

T_n Structures: Base Cases

Base Case: T_1

- One internal vertex c between a and b
- Both a and b are neighbors of c
- Any chord from c is impossible
- Therefore: c is a safe vertex

Base Case: T_2

- Vertices c and d between a and b
- If c has a chord, it must be with b (only non-neighbor)
 - Creates T_1 structure containing d , making d safe
- If d has a chord, it must be with a
 - Creates T_1 structure containing c , making c safe

T_n Structures: General Case Algorithm

Finding a Safe Vertex in T_n in $O(n)$ time:

- ① Consider vertex $a + 1$
- ② $a + 1$ cannot create chords with neighbors a and $a + 2$
- ③ Possible chord partners: $\{a + 3, \dots, b\}$
- ④ Check from b downward using hashset ($O(1)$ per check)

Three outcomes:

- If chord $(a + 1, b)$ exists: Structure becomes T_{n-1} with vertices $a + 2, \dots, b - 1$
- If chord $(a + 1, b - k)$ exists for some $k > 0$: Structure becomes T_{n-k-1}
- Else: $a + 1$ is a safe vertex (no chords with any candidate)

T_n Structures: Conclusion

In each iteration:

- If chord found: n decreases, eventually reaching T_1 or T_2
- If no chord found: vertex is safe

Since:

- Every check reduces problem size or finds safe vertex
- We've proven T_1 and T_2 contain safe vertices

Therefore:

Every T_n structure contains at least one safe vertex.

Final Conclusion:

- Face with no multi-edges: all n vertices can be safe
- Face with at least one multi-edge: at least two complementary T_n structures, each with at least one safe vertex
- **Every face has at least two safe vertices**

Completeness Summary

We have established:

① Pruning preserves all valid triangulations

- Blocking chords never flip in children
- Duplicate blocking chords make all children invalid
- Safe to prune entire branches

② Safe root triangulations always exist

- Every face has at least two safe vertices
- Can always find valid starting triangulation
- Independent of calculation order

Conclusion

Our algorithm covers ALL valid planar triangulations of any planar biconnected graph.

Time Complexity: Main Theorem

Theorem (Amortized Linear Time Complexity):

The algorithm has amortized $O(1)$ time complexity per triangulation, or amortized linear time complexity overall.

Definition (Building Time):

- Time required to calculate safe root and construct chords from root vertex for a single face

Components:

- Finding a safe vertex: $O(n)$
- Adding all chords from the root: $O(n)$
- Total building time for face with n vertices: $O(n)$

Time Complexity: Lower Bound on Triangulations

Lemma (Lower Bound on Triangulation Count):

For a face with n vertices, there are at least $O(n)$ valid triangulations.

Proof:

- Every face has at least two safe vertices: a and c
- Can reach c -root triangulation from a -root through chord flips
- Each flip produces a valid triangulation

Comparing a -root and c -root:

- a -root: all chords have endpoint a
- c -root: all chords have endpoint c
- Only one chord may be shared: $a-c$
- All other $n - 4$ chords must be different
- Requires $n - 4$ flips $\Rightarrow$ at least $n - 3$ triangulations

Time Complexity: Two-Face Graph

Let n_1 and n_2 be vertices in faces 1 and 2:

Total building time: $B = n_1 + n_1 \cdot n_2$

Total triangulations: $T = n_1 \cdot n_2$

Since $n_2 \geq 3$:

$$n_1 \cdot 1 < n_1 \cdot n_2$$

$$\begin{aligned} B &= n_1 + n_1 \cdot n_2 < n_1 \cdot n_2 + n_1 \cdot n_2 \\ &= 2 \cdot n_1 \cdot n_2 \\ &= 2T \end{aligned}$$

Total time complexity: $< 3T$

Time Complexity: Three-Face Graph

Let n_1, n_2, n_3 be vertices in three faces:

$$B = n_1 + n_1 \cdot n_2 + n_1 \cdot n_2 \cdot n_3$$

$$T = n_1 \cdot n_2 \cdot n_3$$

From two-face case: $n_1 + n_1 \cdot n_2 < 2 \cdot n_1 \cdot n_2$

Therefore:

$$B < 2 \cdot n_1 \cdot n_2 + n_1 \cdot n_2 \cdot n_3$$

Since $n_3 \geq 3$, we have $2 < n_3$:

$$2 \cdot n_1 \cdot n_2 < n_1 \cdot n_2 \cdot n_3$$

$$\begin{aligned} B &< n_1 \cdot n_2 \cdot n_3 + n_1 \cdot n_2 \cdot n_3 \\ &= 2T \end{aligned}$$

Total time complexity: $< 3T$

Time Complexity: General Case

For k faces where each has at least 3 vertices:

$$B = n_1 + n_1 \cdot n_2 + \cdots + n_1 \cdot n_2 \cdots n_k$$

$$T = n_1 \cdot n_2 \cdots n_k$$

Pattern holds by induction:

$$n_1 + n_1 \cdot n_2 + \cdots + n_1 \cdot n_2 \cdots n_{k-1} < 2 \cdot n_1 \cdot n_2 \cdots n_{k-1}$$

Since $n_k \geq 3$:

$$2 \cdot n_1 \cdot n_2 \cdots n_{k-1} < n_1 \cdot n_2 \cdots n_k$$

$$B < 2T$$

Total time complexity: $< 3T$

Corollary

For any biconnected planar graph, algorithm takes total time $< 3 \times$

Space Complexity

Theorem (Linear Space Complexity):

The algorithm has $O(n)$ space complexity, where n is the total number of vertices.

Proof:

When processing face i , must store information for faces $1, \dots, i$:

- **Vertex information:** $n_1 + n_2 + \dots + n_i$ vertices
- **Chord information:** For planar graph, $n - 3$ chords total
- **Hashset for chord lookup:** Current chord information

Analysis:

- Total vertices across all faces: $O(n)$
- Number of chords in planar graph: $O(n)$
- Hashset for current chord information: $O(n)$

Therefore: entire algorithm uses $O(n)$ space complexity.

Conclusion

We have rigorously proven our algorithm:

① Is Complete:

- Finds all valid planar triangulations
- Proper pruning of invalid branches

② Always Has a Starting Point:

- At least two safe root vertices exist in every face
- Independent of calculation order

③ Is Efficient in Time:

- Amortized $O(1)$ time per triangulation
- Amortized linear time overall

④ Is Efficient in Space:

- $O(n)$ space complexity

These properties make the algorithm both theoretically sound and practically efficient for enumerating all planar triangulations of biconnected planar graphs.